

High-Availability MLOps FinTech Platform Using Microservices, Kubernetes, Statistical ML Benchmarking and Intelligent Monitoring

Project Objective

Develop a complete **FinTech risk/fraud-classification platform** combining **Spring Boot microservices, React, Machine Learning, MLflow, FastAPI, Kubernetes, CI/CD, Prometheus, Grafana, statistical benchmarking and uncertainty quantification**. The project will address both **ML performance** and the **scalability, reliability, observability and high availability** of the production system.

Main Tasks

1. Microservices FinTech Architecture

Design a **Spring Boot microservices architecture** for customer, transaction, authentication, risk-management, notification and audit services, with REST APIs, API Gateway and secure inter-service communication.

2. ML Classification & Empirical Benchmarking

Use a financial fraud/risk dataset and benchmark several classification algorithms such as **Logistic Regression, Random Forest, Gradient Boosting/XGBoost, SVM and MLP**. Evaluate Accuracy, Precision, Recall, F1, ROC-AUC, PR-AUC, MCC, calibration and inference time.

3. Statistical Validation

Perform repeated experiments/cross-validation with multiple seeds and apply **Friedman/Wilcoxon tests, Holm correction, confidence intervals and effect sizes** to determine whether performance differences are statistically significant.

4. Uncertainty Quantification

Implement prediction uncertainty using calibration, ensembles, bootstrap or conformal prediction. Integrate uncertainty into the decision process so that uncertain predictions can be transferred to human review rather than automatically triggering high-risk actions.

5. MLflow & MLOps

Use **MLflow** for experiment tracking, metrics, parameters, artifacts and model versions. Implement a reproducible pipeline:

Training → Evaluation → Statistical Validation → Model Registry → Deployment
6. FastAPI Model Serving

Deploy the selected model using **FastAPI**, returning prediction, probability, uncertainty and model version. Integrate the ML service with the Spring Boot risk-management service.

7. Data Visualization

Use **PCA, t-SNE and/or UMAP** to visualize financial data, clusters, outliers, class separation, errors and model behavior. Integrate the results into the React dashboard.

8. Docker & Kubernetes

Containerize all components and deploy them on **Kubernetes**,

implementing: • Deployments and Services

- Ingress
- Multiple replicas
- Kubernetes load balancing
- ConfigMaps/Secrets
- Health checks
- Liveness/readiness probes
- Resource limits
- Horizontal Pod Autoscaler (HPA)
- Rolling updates and rollback

9. Load, Overload & High-Availability Experiments

Use **k6, JMeter or Locust** to generate controlled traffic and simulate:

Normal Load → High Load → Overload → Recovery

Measure throughput, latency, error rate, CPU/memory utilization, number of replicas, scaling time and recovery time.

Students will also deliberately terminate pods/services to verify **automatic recovery and service continuity**.

10. Monitoring, Supervision & Observability

Implement a complete monitoring architecture using **Prometheus +**

Grafana. Prometheus will collect metrics from:

- Kubernetes nodes and pods
- Spring Boot microservices
- FastAPI
- Databases
- CPU/memory/network
- Request rate
- Response latency
- HTTP errors
- Number of replicas
- HPA/autoscaling activity
- ML inference time

Grafana will provide dashboards for:

- **System health**
- **Microservice performance**
- **Kubernetes cluster**
- **Load and scalability**
- **High availability**
- **ML inference**
- **Model performance**
- **Prediction uncertainty**
- **Errors and alerts**

Students should configure **threshold-based alerts**, for example:

CPU > 80% → Alert

Response time > 2 s → Alert

Error rate > 5% → Alert

ML uncertainty high → Human review

Pod failure → Alert

Service unavailable → Critical alert

This allows the project to demonstrate **continuous supervision and observability**, rather than simply deploying the application.

11. CI/CD & DevSecOps

Implement:

Git → Build → Tests → SonarQube → Security Scan → Docker → Kubernetes → Integration Tests

with automated deployment and rollback.

Expected Outcome

The final deliverable will be a **complete production-oriented FinTech MLOps platform** demonstrating:

Machine Learning + Statistical Evidence + Uncertainty Quantification + Microservices + MLOps + FastAPI + Docker + Kubernetes + Load Balancing + Auto-scaling + High Availability + CI/CD + Prometheus + Grafana + React.

A GitHub , a full report in markdown format (readme), and a demo of 5 min maximum that explain all the steps followed during the project establishment.

Deadline must be respected (10/01/2027).

5 students by team.